

TD n°3 : Régression Logistique

L'objectif de ce TP est de mettre en pratique les outils associés à Python pour réaliser une Régression Logistique. Si les algorithmes de régression vus dans le cadre des deux TDs précédents servent à tenter de *prédire une valeur* à partir d'un ensemble de variables explicatives, la régression logistique tente, elle, de *prédire à quelle catégorie se rattache un ensemble de valeurs*. Il s'agit donc d'un algorithme de classification, connu et très employé, en apprentissage supervisé. Nous allons mettre en pratique cet algorithme, en utilisant Python et la bibliothèque Scikit-Learn. **A rendre :** pour chaque exercice, code et affichage (lorsque c'est possible), le tout, au choix, sous la forme d'un notebook jupyter, ou d'un fichier .pdf

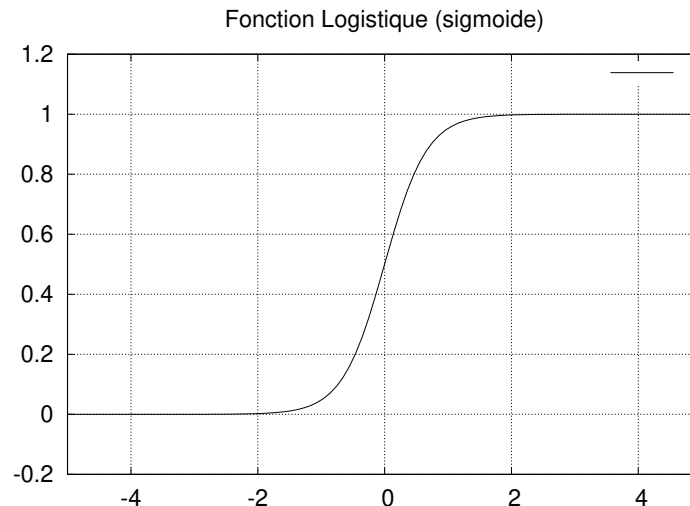
Principe. Fondamentalement, un modèle de Régression Logistique a pour but de prédire la probabilité qu'un évènement arrive (valeur 1) ou pas (valeur 0) à partir de l'*optimisation des coefficients de régression*. Ce résultat varie toujours entre 0 et 1. Lorsque la valeur prédite est supérieure à un seuil, l'évènement est susceptible de se produire, alors que lorsque cette valeur est inférieure au même seuil, il ne l'est pas. Mathématiquement, à partir d'un vecteur d'entrée $X = (x_1, x_2, \dots, x_n)$, la Régression Logistique a pour objectif de trouver une fonction h_X telle qu'il soit possible de calculer

$$y = \{1 \text{ si } h_X \geq \text{seuil}, 0 \text{ si } h_X < \text{seuil}\}$$

Autrement dit, on attend de la fonction h_X qu'elle soit une mesure de probabilité (*i.e.* comprise entre 0 et 1), paramétrée par $x_1, x_2, \dots, x_n$ à optimiser, et que le *seuil* dont il est question corresponde au critère de discrimination qu'on souhaite considérer (généralement valant 0.5). La fonction qui remplit le mieux ces condition est la fonction *sigmoïde*, définie sur $\mathbb{R}$ à valeurs dans $[0, 1]$, qui s'écrit :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Graphiquement, elle correspond à une courbe en forme de S qui a pour limites 0 et 1 lorsque x tend respectivement vers $-\infty$ et $+\infty$, en passant par $y = 0.5$ en $x = 0$.



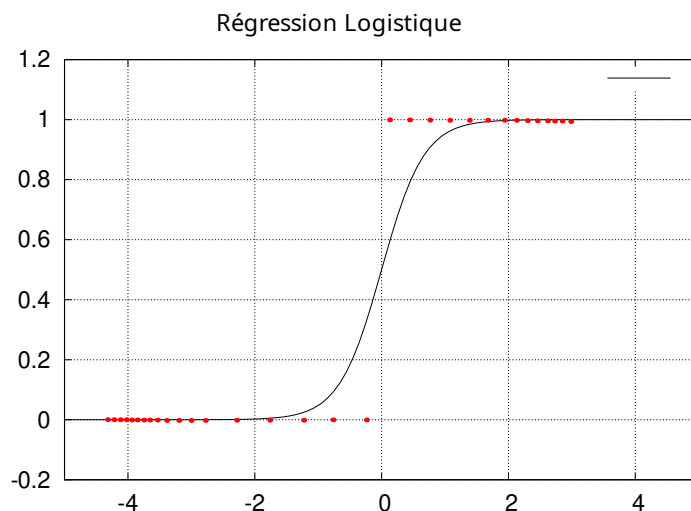
La fonction h qui définit la Régression Logistique s'écrit alors :

$$(\forall X)(X \in \mathbb{R}^n)(h(X) = \sigma(\Theta X))$$

autrement dit :

$$(\forall X)(X \in \mathbb{R}^n)(h(X) = \frac{1}{1 + e^{-\sum_{i=1}^n \theta_i \cdot x_i}})$$

Un problème de classification par Régression Logistique se ramène donc à un problème d'optimisation dans lequel, à partir des données, on tente d'obtenir le jeu de paramètres Θ_i le plus adéquat (permettant à la courbe sigmoïde de coller au mieux aux données). C'est à cette étape qu'intervient la phase l'apprentissage. Une fois cette étape effectuée, voici un aperçu de ce qui peut être obtenu :



Il ne reste plus, à partir du seuil défini, qu'à classer les points en fonction de leurs positions par rapport à la régression.

Jeu de données utilisé. Pour ce tutoriel, on utilisera un jeu de données célèbre pour les problèmes d'apprentissage, représenté par une base de données regroupant les caractéristiques de trois espèces de fleurs d'Iris, à savoir *Setosa*, *Versicolour* et *Virginica*. Chaque ligne de ce jeu de données est une observation des caractéristiques d'une fleur d'Iris. Ce dataset décrit les trois espèces d'Iris considérées par quatre propriétés : longueur et largeur de sépales ainsi que longueur et largeur de pétales. La base de données comporte 150 observations (50 observations par espèce). Pour plus d'informations, voir l'url https://fr.wikipedia.org/wiki/Iris_de_Fisher qui fournit des informations abondantes sur ce jeu de données (éventuellement disponible aussi sur *Ametice* avec ce sujet).

Exercice 0. Implémentation de la régression logistique sur Iris.

1. Importez les bibliothèques `numpy`, `pyplot` (de Matplotlib) et `sklearn` (qui est le nom de Scikit-Learn).
2. Chargez le package `datasets` de `sklearn` pour retrouver le jeu de données IRIS. Scikit-Learn vient avec un ensemble de jeu de données prêt à l'emploi pour des fins d'expérimentation. Ces dataset sont regroupés dans le package `sklearn.datasets`. Alternativement, et comme signalé plus haut, vous pouvez récupérer le dataset correspondant sur *Ametice* avec ce sujet (mais c'est un peu moins bien...).
3. Chargez le jeu de données Iris : on utilise la méthode `load_iris()` du package `datasets`.
4. Affichez à l'écran le jeu de données Iris.

Exercice 1. Adaptation de la base de données Iris. Comme évoqué précédemment, le dataset Iris se compose de quatre *features* (variables explicatives ou prédictives). Pour simplifier, on n'utilisera que les deux premières variables explicatives à savoir : `Sepal_length` et `Sepal_width`. De cette façon, on fera de la régression logistique dans un espace bidimensionnel. Le jeu de données Iris se compose des trois classes *Setosa*, *Versicolour* et *Virginica* auxquelles il est possible d'associer des étiquettes appartenant à l'ensemble $\{0, 1, 2\}$. Il s'agit donc d'une catégorisation multi-classes. La régression logistique étant un algorithme de classification binaire, on va re-étiqueter les fleurs ayant le label (étiquette) 1 et 2 avec le label 1. Ainsi, on se ramène ainsi à un problème de classification binaire.

1. Prendre pour x les deux premières colonnes du jeu de données Iris (utiliser la méthode `data`).
2. Prendre pour y le target (cible) du jeu de données Iris, en re-étiquetant les fleurs ayant les label 1 et 2 avec le seul label 1 (utiliser la méthode `target`).

Exercice 2. Visualisation du jeu de données. Utilisez `pyplot` pour visualiser les valeurs des deux variables explicatives du tableau x , en utilisant du vert pour les points avec l'étiquette 0 et du jaune pour les points avec l'étiquette 1. On remarque que les données de la classe 0 et la classe 1 peuvent être linéairement séparées : une ligne droite permet de distinguer les points appartenant à la classe 0 et les points appartenant à la classe 1 dans le plan cartésien des deux variables explicatives. Une régression logistique doit donc permettre de départager les deux classes.

Exercice 3. Entraînement d'un modèle de régression logistique. Scikit-Learn offre une classe d'implémentation de la Régression Logistique. Oninstanciera cette classe pour entraîner un modèle prédictif.

1. Importez la classe `LogisticRegression` de la bibliothèque `linear_model` de `sklearn`.
2. Créez un modèle pour la Régression Logistique, c'est-à-dire une objet de la classe `LogisticRegression`, avec le paramètre `C=1e20`.
3. Entraînez le modèle en utilisant la méthode `fit(x,y)` de la classe `LogisticRegression`, où la variable x est l'ensemble des observations qui serviront à l'entraînement de l'algorithme de Machine Learning (c'est-à-dire, les variables explicatives), et y est l'ensemble des étiquettes données aux observations.

Exercice 4. Prédire à quelle classe appartiennent de nouvelles fleurs d'Iris. Maintenant qu'on a entraîné notre algorithme de Régression Logistique, on va l'utiliser pour prédire la classe de fleurs d'Iris qui ne figuraient pas dans le jeu d'entraînement. Pour rappel : seules sont utilisées les variables explicatives `Sepal_Length` et `Sepal_Width` pour entraîner notre jeu de données. On fournit des valeurs pour ces deux variables, à partir desquelles le modèle prédictif devra être capable de déterminer la classe d'une nouvelle fleur.

1. Créez un tableau avec 4 nouvelles observations (c'est-à-dire, 4 nouvelles valeurs pour les variables explicatives).
2. Affichez les nouvelles observations en utilisant `pyplot`, avec une couleur différente de celles des données du jeu Iris.
3. Utilisez la méthode `predict()` de la classe `LogisticRegression` pour prédire à quelle classe les nouvelles observations appartiennent.
4. Vérifiez visuellement que l'étiquetage donné aux nouvelles observations et compatible avec les jeu de donnée Iris que vous avez visualisé dans l'exercice 2. Par exemple, pour le tableau suivant :

```
Iries_to_Predict = [  
    [5.5, 2.5],  
    [7,3],  
    [3,2],  
    [5,3]  
]
```

la méthode `predict()` appliquée à `Iries_To_Predict` devra être capable d'associer les deux premières observations à la classe 1, et les deux dernières observations à la classe 0.

Exercice 5. Régression Logistique multi-classes. Telle qu'on l'a vue jusqu'ici, la Régression Logistique permet seulement de procéder à une classification binaire des observations (Spam/Non SPAM, MALIGNE/-BENIGNE, NOIR/BLANC . . .) ce qui est assez contraignant ! Le jeu de données Iris d'origine catégorise les fleurs d'iris en 3 classes. Comment faire de la Régression Logistique sur un nombre de classes supérieur à 2 (autrement dit dans un cadre multi-classes) ? L'idée est d'itérer la Régression Logistique (binaire) plusieurs fois. Le principe est simple : il consiste à découper le problème de classification multi-classes en autant de problèmes de classification binaire que nécessaires.

1. Répétez les exercices 1-4, mais cette fois en re-étiquetant les fleurs ayant les labels (étiquettes) 0 et 2 avec le label 0.
2. Répétez les exercices 1-4, mais cette fois en re-étiquetant les fleurs ayant les labels (étiquettes) 0 et 1 avec le label 1.

A quelle classe appartiennent les nouvelles observations de l'exercice 4 ?